ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
**KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**




## CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT - CO2003

---

## BÀI TẬP LỚN 2

# BKMove: TÌM TRẠM XE BUÝT VÀ BẢNG GIỜ XE ĐẾN

---

Thành phố Hồ Chí Minh, 10/2026

# ĐẶC TẢ BÀI TẬP LỚN

**Phiên bản 1.0**

## 1 Kết quả đạt được

Sau khi hoàn thành bài tập lớn, sinh viên có thể:

- Hiện thực các thao tác cơ bản trên Binary Search Tree (BST), AVL Tree và Binary Heap.
- Duy trì đúng các quy tắc tổ chức dữ liệu khi thêm, xóa và cập nhật phần tử.
- Kết hợp hoặc mở rộng những cấu trúc đã học để thực hiện tìm kiếm, phân trang và quản lý các lượt xe đến.
- Kiểm tra kết quả, phân tích độ phức tạp và quản lý bộ nhớ của chương trình C++.

## 2 Giới thiệu

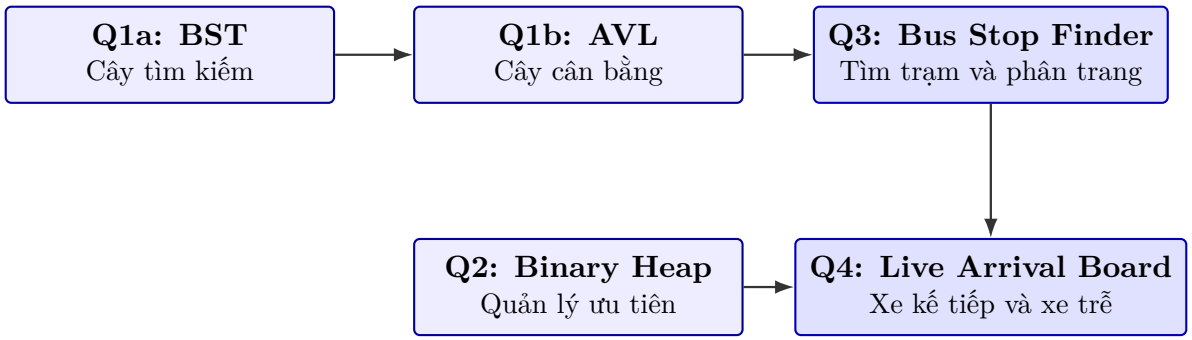
BKMove là ứng dụng giao thông công cộng được giới thiệu trong BTL 1. Ở bài tập lớn này, sinh viên xây dựng **hai chức năng khác** của BKMove: tìm trạm xe buýt theo tên và theo dõi những lượt xe sắp đến tại một trạm.

**BTL 2 là một project riêng.** Sinh viên sử dụng starter code của BTL 2; không cần sao chép `BusRoute` hoặc bất kỳ file bài làm nào từ BTL 1. Dữ liệu và giao diện của hai bài tập được thiết kế độc lập.

Bài tập lớn gồm năm phần, mỗi phần **2 điểm**:

- **Câu 1a – BST:** Thêm, tìm kiếm và xóa khóa trong cây tìm kiếm nhị phân.
- **Câu 1b – AVL:** Giữ cây tìm kiếm cân bằng sau khi thêm và xóa.
- **Câu 2 – Binary Heap:** Hiện thực hàng đợi ưu tiên trên mảng.
- **Câu 3 – Bus Stop Finder:** Tìm trạm và xem danh sách theo từng trang.
- **Câu 4 – Live Arrival Board:** Quản lý các lượt xe đến và cập nhật giờ đến dự kiến.

Câu 1a cung cấp BST nền cho Câu 1b. Câu 3 sử dụng hoặc mở rộng các cấu trúc tìm kiếm. Câu 4 sử dụng dữ liệu trạm của Câu 3 và có thể tận dụng Heap từ Câu 2. Tổng điểm là **10 điểm**. Hình 1 tóm tắt mối liên hệ giữa năm phần.



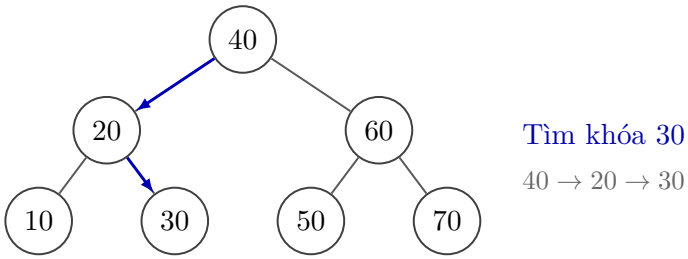

Hình 1: Năm phần đánh giá và quan hệ phụ thuộc về mặt khái niệm.

# 3 Kiến thức nền và mô hình ứng dụng

Các thuật toán trong phần này đã được trình bày ở lớp. Những hình dưới đây nhắc lại dạng dữ liệu mà sinh viên sẽ hiện thực trong ba phần đầu.

## 3.1 Binary Search Tree (BST)

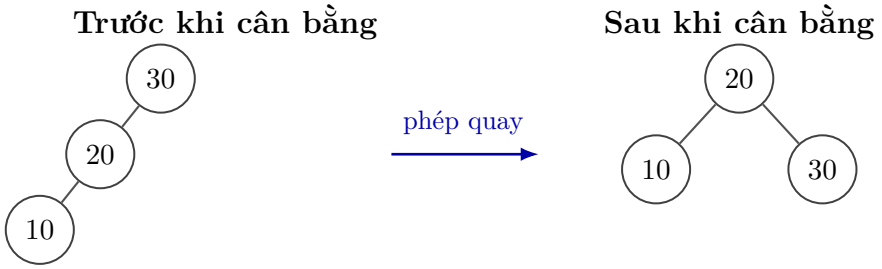
BST sắp xếp các khóa sao cho khóa ở cây con trái nhỏ hơn khóa của node cha, còn khóa ở cây con phải lớn hơn. Cấu trúc này cho phép tìm kiếm theo đường đi từ gốc; thời gian phụ thuộc vào chiều cao cây. Hình 2 minh họa đường tìm khóa 30 qua các nhánh của BST.




Hình 2: Cây tìm kiếm nhị phân và đường đi khi tìm khóa 30.

## 3.2 AVL Tree

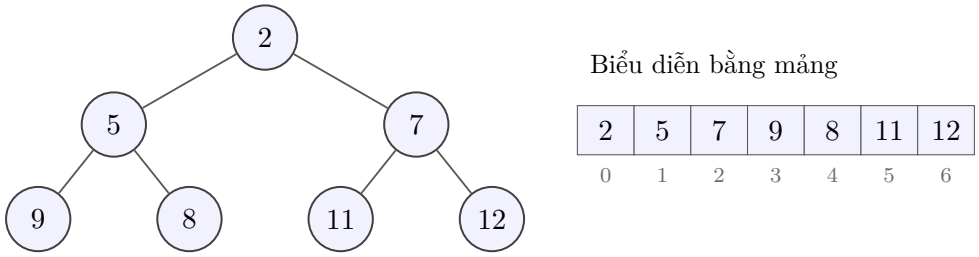
AVL là BST có cân bằng chiều cao. Sau thao tác thêm hoặc xóa, các phép quay có thể được sử dụng để khôi phục cân bằng mà vẫn giữ nguyên thứ tự khóa. Hình 3 minh họa phép quay phải để cân bằng cây.




Hình 3: Một phép quay phải khôi phục cân bằng nhưng vẫn bảo toàn thứ tự khóa khi duyệt inorder.

### 3.3 Binary Heap

Binary Heap được biểu diễn bằng cây nhị phân hoàn chỉnh, thường lưu bằng mảng. Với Min-Heap, phần tử có độ ưu tiên nhỏ nhất nằm tại gốc; một comparator có thể thay đổi cách xác định thứ tự ưu tiên. Hình 4 đối chiếu dạng cây và mảng, với phần tử nhỏ nhất ở chỉ số 0.




Hình 4: Min-Heap dưới dạng cây nhị phân hoàn chỉnh và mảng.

## 4 Yêu cầu

Sinh viên hoàn thành các phương thức còn thiếu trong starter code. Các chữ ký public được cung cấp phải được giữ nguyên. Sinh viên được bổ sung thành phần nội bộ trong **năm file bài làm** đã quy định ở Mục 7.

### 4.1 Câu 1a – Lớp `BST<K,V>` (2 điểm)

**File:** `BST.h`.

Hiện thực các thao tác thêm, tìm và xóa trên BST. Mỗi khóa chỉ được lưu một lần; thêm khóa đã tồn tại không được thay đổi giá trị cũ. Xóa phải xử lý node lá, node có một con và node có hai con. Các thao tác `size()`, `height()` và `clear()` phải phản ánh đúng trạng thái cây.

```
void add(K key, V value = 0);
V remove(K key, bool* success = 0);
V search(K key, bool& found);
int size();
int height();
void clear();
```

Trong bài này, BST được sử dụng với `V` là kiểu con trỏ. `remove()` chuyển quyền sở hữu giá trị được trả về cho bên gọi: xóa node không đồng nghĩa với xóa giá trị đó. Nếu không tìm thấy khóa, `search()` và `remove()` trả về con trỏ null; đối số `success` của `remove()` có thể không được truyền vào. Nếu thêm khóa trùng, giá trị không được chèn vẫn thuộc bên gọi.

**Độ phức tạp:** thêm, tìm và xóa chạy trong $O(h)$, với $h$ là chiều cao BST. Câu này không yêu cầu cân bằng cây.

## 4.2 Câu 1b – Lớp `AVL<K,V>` (2 điểm)

**File:** `AVL.h`.

Hoàn thiện AVL dựa trên BST của Câu 1a. Thêm và xóa phải duy trì thứ tự BST, cập nhật chiều cao và khôi phục cân bằng khi cần. Hiện thực đủ các tình huống quay LL, RR, LR và RL. Những quy tắc về khóa trùng, cây rỗng và quản lý giá trị con trỏ được giữ như Câu 1a.

**Độ phức tạp:** thêm, tìm và xóa phải đạt $O(\log n)$ trong trường hợp xấu nhất.

## 4.3 Câu 2 – Lớp `Heap<T>` (2 điểm)

**File:** `Heap.h`.

Hiện thực các thao tác trên Binary Heap được cung cấp. `push()` thêm phần tử, `peek()` xem phần tử ưu tiên nhất mà không xóa, `pop()` lấy và xóa phần tử đó. `remove(item)` chỉ xóa **một** phần tử khớp; nếu có nhiều giá trị bằng nhau, những phần tử còn lại vẫn ở trong Heap. `heapify()` thay nội dung Heap bằng những phần tử từ mảng đầu vào.

```
void push(T item);
const T peek();
T pop();
void remove(T item, void (*removeItemData)(T) = 0);
void heapify(T array[], int size);
```

Khi Heap rỗng, `peek()` và `pop()` ném `std::underflow_error`. Nếu không có comparator, Heap dùng thứ tự nhỏ nhất trước. Nếu `remove()` tìm được phần tử và có callback

`removeItemData`, callback được gọi đúng một lần cho phần tử bị xóa; không tìm thấy thì không gọi callback và Heap giữ nguyên.

`heapify()` phải sử dụng cách xây Heap bottom-up trong $O(m)$, không được gọi `push()` lần lượt $m$ lần. Các thao tác `peek()`, `push()`, `pop()` lần lượt đạt $O(1)$, $O(\log m)$ và $O(\log m)$; `remove(item)` không bắt buộc nhanh hơn tuyến tính.

## 4.4 BKMove

Trong BKMove, hành khách có thể tìm trạm bằng tên, chọn một trạm từ kết quả rồi xem lượt xe nào dự kiến đến sớm nhất. Nếu một lượt xe bị trễ hoặc bị hủy, thông tin hiển thị cần được cập nhật tương ứng.

Mỗi đối tượng `BKMove` quản lý **danh mục trạm** và **các lượt xe còn chờ** tại những trạm đó. Đối tượng mới tạo ban đầu rỗng. Một lượt xe được giữ trong lịch chờ cho đến khi hệ thống xử lý hoặc hủy lượt xe đó. Vì thế, trạm có lượt xe đang chờ không thể bị xóa khỏi danh mục.

Hai chức năng tiếp theo được hiện thực trong `BKMove.h` và `BKMove.cpp`. Các kiểu dữ liệu và public API đã được cung cấp; sinh viên tự đề xuất các biến thành viên `private` và cách tổ chức dữ liệu bên trong lớp.

## 4.5 Câu 3 – BKMove: Bus Stop Finder (2 điểm)

Bus Stop Finder giúp hành khách tìm trạm theo phần đầu tên trạm và xem kết quả trên nhiều trang. Chẳng hạn, nhập `"ben"` sẽ tìm các trạm có khóa tìm kiếm bắt đầu bằng ba ký tự này.

**Thông tin trạm.** Mỗi trạm được biểu diễn bằng kiểu dữ liệu sau trong `BKMove.h`:

```
struct StopInfo {
    std::string stopId;
    std::string displayName;
    std::string searchKey;
};
```

`stopId` là mã định danh duy nhất; `displayName` là tên hiển thị; `searchKey` là chuỗi dùng cho tìm kiếm. Ví dụ, trạm *Bến Thành* có thể có `searchKey = "ben thanh"`. Khóa tìm kiếm và tiền tố đầu vào đã được chuẩn hóa sẵn; sinh viên không phải tự xử lý dấu tiếng Việt hay Unicode. Các trường của trạm không được rỗng. Nhiều trạm được phép có cùng tên hoặc `searchKey`, nhưng không được trùng `stopId`.

Ví dụ 1 minh họa cách tìm theo tiền tố, sắp xếp và phân trang.

**Ví dụ 1. Tìm trạm và chuyển trang**

Giả sử bảy trạm được thêm vào BKMove theo đúng thứ tự sau (chưa sắp xếp):

| stopId | displayName | searchKey |
|---|---|---|
| S007 | Chợ Lớn | `cho lon` |
| S005 | Bến xe Miền Tây | `ben xe mien tay` |
| S001 | Bến Thành | `ben thanh` |
| S012 | Đại học Bách Khoa | `dai hoc bach khoa` |
| S004 | Bến xe Miền Đông | `ben xe mien dong` |
| S009 | Bến xe Chợ Lớn | `ben xe cho lon` |
| S021 | Công viên Tao Đàn | `cong vien tao dan` |

Khi tìm tiền tố `"ben"`, BKMove chỉ giữ lại bốn trạm có `searchKey` bắt đầu bằng `ben`. Các kết quả được sắp theo cặp `(searchKey, stopId)` tăng dần, *không* theo thứ tự thêm vào:

```
S001 -> S009 -> S004 -> S005
```

Nếu mỗi trang có hai kết quả thì:

| Lời gọi | Kết quả theo thứ tự |
|---|---|
| `findStops("ben", 0, 2)` | `[S001, S009]` |
| `findStops("ben", 1, 2)` | `[S004, S005]` |
| `findStops("ben", 2, 2)` | Vector rỗng |
| `findStops("ben xe", 0, 2)` | `[S009, S004]` |

`page = 0` là trang đầu tiên, còn `page = 1` là trang thứ hai. Hình 5 đánh dấu hai trạm được trả về ở trang thứ hai.

**Tìm trạm: tiền tố = ben** — Bốn kết quả phù hợp

| Vị trí | Tên trạm | stopId |
|---|---|---|
| 0 | Bến Thành | S001 |
| 1 | Bến xe Chợ Lớn | S009 |
| 2 | Bến xe Miền Đông | S004 |
| 3 | Bến xe Miền Tây | S005 |

Yêu cầu: page = 1, pageSize = 2 (đếm từ 0)
Bản ghi trả về: vị trí 2 và 3

Hình 5: Truy cập trực tiếp trang thứ hai của danh sách trạm phù hợp; các dòng tô xanh là kết quả.

Ví dụ 2 minh họa cách dùng `stopId` để sắp xếp khi các trạm trùng khóa tìm kiếm.

**Ví dụ 2. Hai trạm trùng tên**

Nếu BKMove có hai trạm cùng mang tên *Bến Thành*, có cùng `searchKey = "ben thanh"`, nhưng mã là `S001` và `S010`, cả hai vẫn phải xuất hiện. Khi sắp xếp, `S001` đứng trước `S010` do quy tắc so sánh `stopId`. Ngược lại, một lệnh thêm trạm khác mang mã `S001` phải thất bại và không sửa bản ghi đang có.

**Các phương thức cần hiện thực** (đúng như khai báo trong `BKMove.h`):

```
bool addStop(const StopInfo& stop);
bool removeStop(const std::string& stopId);
bool findStop(const std::string& stopId, StopInfo& out) const;
int getStopCount() const;
std::vector<StopInfo> findStops(
    const std::string& prefix, int page, int pageSize) const;
```

1. **`addStop(stop)`:** Thêm trạm nếu các trường hợp lệ và chưa có `stopId` đó; trả về `true` khi thành công. Thất bại trả về `false`, dữ liệu giữ nguyên.
2. **`removeStop(stopId)`:** Xóa trạm theo ID; trả về `false` nếu trạm không tồn tại hoặc còn lượt xe đang chờ tại chính trạm đó. Xóa thành công phải loại trạm khỏi mọi kết quả tìm kiếm.
3. **`findStop(stopId, out)`:** Tìm theo ID. Thành công chép trạm vào `out`; thất bại trả về `false` và giữ `out` không đổi.
4. **`getStopCount()`:** Trả về số trạm đang đăng ký.
5. **`findStops(prefix, page, pageSize)`:** Trả về trang được yêu cầu của các trạm có

`searchKey` bắt đầu bằng `prefix`, sắp theo `(searchKey, stopId)`. Tiền tố rỗng khớp mọi trạm. Trang âm, `pageSize <= 0` hoặc trang vượt ngoài kết quả trả vector rỗng.

**Yêu cầu về hiệu năng.** Giả sử danh mục có 100.000 trạm, mỗi trang hiển thị 10 kết quả. Nếu người dùng chọn ngay `page = 5000` với `prefix = ""`, chương trình cần lấy 10 trạm bắt đầu từ vị trí 50.001. Duyệt qua 50.000 trạm đứng trước vẫn cho kết quả đúng, nhưng sẽ chậm khi người dùng chọn các trang ở rất xa trang đầu.

Vì vậy, với $n$ là tổng số trạm và $r$ là số trạm thực sự được trả về, hai thao tác thêm và xóa phải đạt $O(\log n)$; thao tác lấy một trang bằng `findStops()` phải đạt $O(\log n + r)$, **không phụ thuộc tuyến tính vào số phần tử của các trang trước**. Chi phí so sánh khóa được xem là hằng số theo quy ước của bài tập.

**Gợi ý.** AVL tìm kiếm rất hiệu quả theo khóa. Tuy nhiên, tìm một khóa cụ thể và truy cập phần tử nằm ở vị trí thứ 50.001 trong danh sách đã sắp xếp là hai yêu cầu khác nhau. Có thể tận dụng thông tin về những phần dữ liệu không cần duyệt để đi đến đúng trang nhanh hơn không?

## 4.6 Câu 4 – BKMove: Live Arrival Board (2 điểm)

Khi hành khách đã chọn một trạm, BKMove hiển thị lượt xe dự kiến đến sớm nhất. Giờ đến có thể được cập nhật khi xe trễ; một lượt xe mới cũng có thể được thêm vào hoặc bị hủy khỏi lịch.

**Thông tin lượt xe.** Mỗi lượt được biểu diễn như sau:

```
struct ArrivalEvent {
    std::string eventId;
    std::string stopId;
    std::string routeId;
    int arrivalTime;
};
```

`eventId` nhận diện một lượt xe đang chờ trong BKMove, `stopId` chỉ trạm, `routeId` là mã tuyến dùng để hiển thị. `arrivalTime` là số nguyên không âm trên *cùng một trục thời gian giả lập*; nó không phải đồng hồ thực và không tự thay đổi khi thời gian trôi qua. Event chỉ rời trạng thái đang chờ khi được `process` hoặc `cancel`.

Ví dụ 3 cho thấy việc đổi giờ đến có thể thay đổi lượt xe sớm nhất.

**Ví dụ 3. Xe bị trễ**

Trước tiên, BKMove đã có trạm `S012` (Đại học Bách Khoa). Hệ thống đăng ký ba lượt xe:

```
bk.scheduleArrival({"E01", "S012", "08", 5});
bk.scheduleArrival({"E02", "S012", "19", 8});
bk.scheduleArrival({"E03", "S012", "14", 12});
```

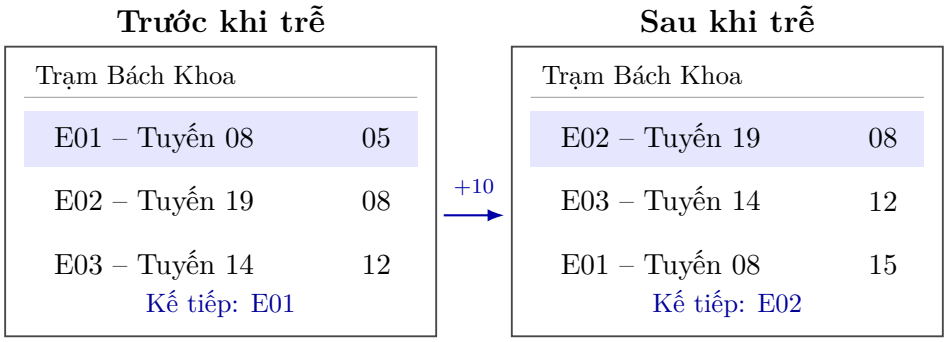
Lệnh `peekNextArrival("S012", out)` trả về `E01`, bởi thời điểm 5 nhỏ nhất. `peek` chỉ xem nên gọi lại vẫn trả về `E01`.

Sau đó, xe `E01` bị trễ và giờ đến được đổi thành 15:

```
bk.rescheduleArrival("E01", 15);
```

Thứ tự mới là `E02 (8), E03 (12), E01 (15)`. Lúc này, `peekNextArrival("S012", out)` phải trả về `E02`.

Hình 6 đối chiếu thứ tự xe trước và sau lần đổi giờ đến này.




Hình 6: Đổi giờ đến giả lập của E01 từ 05 sang 15 khiến xe tiếp theo thay đổi từ E01 thành E02.

Ví dụ 4 tiếp tục với thao tác xử lý, hủy và xóa trạm.

**Ví dụ 4. Xử lý, hủy và xóa trạm**

Bắt đầu từ trạng thái sau Ví dụ 3, các thao tác lần lượt có kết quả:

| Thao tác | Kết quả |
|---|---|
| `processNextArrival("S012", out)` | Xử lý E02; còn E03 và E01. |
| `removeStop("S012")` | `false`: vẫn còn hai event. |
| `cancelArrival("E03")` | Hủy E03; vẫn còn E01. |
| `removeStop("S012")` | Vẫn `false`. |
| `cancelArrival("E01")` | Trạm không còn event đang chờ. |
| `removeStop("S012")` | `true`: xóa trạm thành công. |

Xử lý một lượt xe không đồng nghĩa với việc trạm đã hết lịch chờ. Điều kiện xóa trạm chỉ được thỏa mãn sau khi *tất cả* event của trạm được xử lý hoặc hủy. Lượt xe ở trạm khác không ảnh hưởng đến việc xóa `S012`.

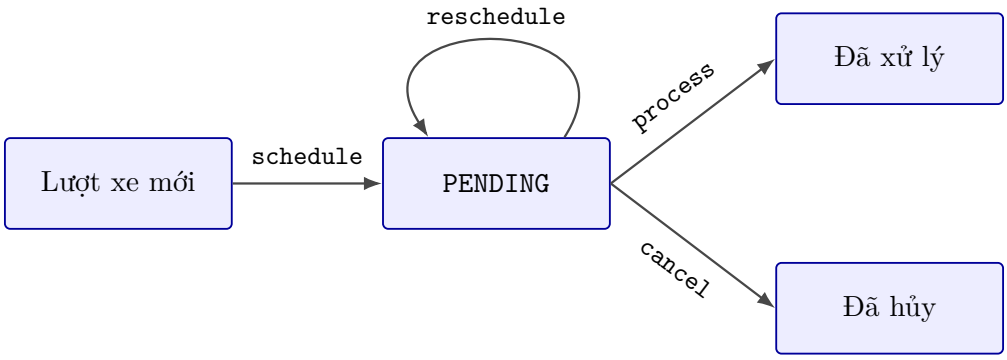
**Các phương thức cần hiện thực:**

```
bool scheduleArrival(const ArrivalEvent& event);
bool peekNextArrival(const std::string& stopId,
                     ArrivalEvent& out) const;
bool processNextArrival(const std::string& stopId,
                        ArrivalEvent& out);
bool rescheduleArrival(const std::string& eventId,
                       int newArrivalTime);
bool cancelArrival(const std::string& eventId);
```

1. **`scheduleArrival(event)`:** Thêm event đang chờ khi `eventId`, `stopId`, `routeId` không rỗng, `arrivalTime >= 0`, trạm tồn tại và `eventId` chưa được dùng bởi event đang chờ nào trong BKMove. Không đạt các điều kiện thì trả về `false` và không đổi trạng thái.
2. **`peekNextArrival(stopId, out)`:** Xem event sớm nhất ở trạm; không xóa. Nếu trạm không có event đang chờ hoặc trạm không tồn tại, trả về `false`, giữ `out` không đổi.
3. **`processNextArrival(stopId, out)`:** Lấy và xóa event sớm nhất của trạm. Khi thất bại, `out` không đổi.
4. **`rescheduleArrival(eventId, newArrivalTime)`:** Đổi giờ đến của event đang chờ sang một giá trị không âm, sớm hơn hoặc muộn hơn. Không đổi trạm hay tuyến và không xóa event. Nếu ID không tồn tại hoặc giờ mới âm, trả về `false` và giữ nguyên trạng thái.
5. **`cancelArrival(eventId)`:** Xóa event đang chờ theo ID, kể cả khi event đó không phải lượt sớm nhất. Nếu ID không tồn tại thì trả về `false`. ID của event đã xử lý hoặc hủy được phép dùng lại cho lượt xe mới.

Khi nhiều event có cùng `arrivalTime`, event với `eventId` nhỏ hơn theo thứ tự từ điển được ưu tiên. Ví dụ `E06` đứng trước `E07` khi hai lượt cùng có thời điểm 10. Quy tắc này áp dụng trong mỗi trạm. `eventId` phải duy nhất trên *toàn bộ* BKMove khi event còn đang chờ.

Hình 7 cho thấy event chỉ hết chờ khi được xử lý hoặc hủy.




Hình 7: Một lượt xe vẫn ở trạng thái chờ sau khi đổi ETA; xử lý hoặc hủy sẽ loại lượt xe đó.

**Yêu cầu về hiệu năng.** Heap giúp xem phần tử ưu tiên nhất nhanh, nhưng khi `E01` bị trễ hoặc `E03` bị hủy, lời gọi chỉ cho biết `eventId`, không cho vị trí của event trong Heap. Nếu phải quét toàn bộ lịch mỗi lần sửa, thời gian sẽ tăng theo số lượt xe đang chờ.

Gọi $n$ là số trạm và $m$ là tổng số event đang chờ. Các phương thức phải đáp ứng:

- `peekNextArrival()`: $O(\log n)$.
- `scheduleArrival()` và `processNextArrival()`: $O(\log n + \log m)$.
- `rescheduleArrival()` và `cancelArrival()`: $O(\log m)$.

Các mức trên áp dụng cho *toàn bộ* phương thức BKMove, bao gồm cả bước tìm trạm khi cần. Với mảng động, chi phí cấp phát lại được phép xét theo trung bình trên nhiều thao tác (amortized).

**Gợi ý.** Heap cho biết lượt xe sớm nhất, nhưng không tự tìm nhanh một event theo ID. Có thể kết hợp các cấu trúc đã học để định vị event, thay đổi thời gian và sắp xếp lại lượt xe mà không phải xem lần lượt mọi event không?

# 5 Public tests và các mốc kiểm tra

Starter cung cấp **51 public test case**, chia theo năm phần: Q1a (9), Q1b (10), Q2 (10), Q3 (10) và Q4 (12). Sinh viên có thể kiểm tra từng phần trước khi hoàn thành cả bài:

```
make q1a   make q1b   make q2   make q3   make q4
```

Chạy tất cả bằng `make all`; chọn một case cụ thể, chẳng hạn `make q3 TEST=4`; kiểm tra việc sử dụng thư viện bằng `make check-rules`. Các public tests không bao phủ tất cả dữ liệu hoặc

trường hợp biên. Hidden tests có thể kiểm tra thêm tính đúng đắn, quản lý bộ nhớ và hiệu năng theo đặc tả.

**Chấm điểm:** Q1a, Q1b và Q2 mỗi phần tối đa 2 điểm. Q3 và Q4 mỗi phần gồm 1 điểm cho chức năng và 1 điểm cho yêu cầu hiệu năng; kết quả chức năng phải đủ đúng để đánh giá phần hiệu năng. Việc chạy nhanh trên một bộ test riêng lẻ không tự chứng minh độ phức tạp thuật toán.

# 6 Hướng dẫn

Để thực hiện bài tập lớn, sinh viên sử dụng starter code đi kèm với tài liệu này:

1. **Chuẩn bị.** Giải nén starter package và mở thư mục `init/`. Có thể chạy `make demo` để biên dịch chương trình ví dụ và kiểm tra môi trường C++17 trước khi bắt đầu.
2. **Các file bài làm.** `BST.h` (Q1a), `AVL.h` (Q1b), `Heap.h` (Q2), `BKMove.h` và `BKMove.cpp` (Q3–Q4). Chỉ năm file này được lấy từ bài nộp khi chấm.
3. **Thứ tự thực hiện.** Hoàn thành BST trước AVL; sau đó Heap. Q3 quản lý danh mục trạm, còn Q4 sử dụng các trạm đã được đăng ký để đặt lịch xe. Vì thế, nếu Q3 chưa hoạt động, một số test Q4 có thể thất bại ngay ở bước thêm trạm. Sinh viên có thể kiểm thử từng câu bằng lệnh tương ứng.
4. **Phạm vi chỉnh sửa.** Sinh viên được bổ sung biến `private`, helper, `struct`, `class` hoặc lớp kế thừa trong đúng năm file nêu trên. Có thể mở rộng các cấu trúc BST, AVL, Heap để đáp ứng yêu cầu Q3/Q4. Không thay đổi public API hoặc các trường dữ liệu gốc của `StopInfo` và `ArrivalEvent`; có thể bổ sung trường mới nếu vẫn giữ nguyên hành vi của những dữ liệu đã cung cấp. `BKMove` không được sao chép trực tiếp vì copy constructor và phép gán copy đã bị vô hiệu hóa.
5. **File được cung cấp.** Thư mục `support/` chứa interface và mã hỗ trợ, `tests/` chứa public tests, `main.cpp` là chương trình ví dụ và `Makefile` định nghĩa các lệnh build/test. Không sửa những file này; mọi thay đổi ở đó sẽ không được lấy sang hệ thống chấm.
6. **Kiểm thử và sửa lỗi.** Chạy `make q1a`, `make q1b`, `make q2`, `make q3`, `make q4` hoặc `make all`. Để cô lập một trường hợp, sử dụng `TEST=` như ví dụ ở Mục 5. Nếu có lỗi con trỏ, sinh viên nên kiểm tra cả trường hợp xóa node, hủy event và giải phóng đối tượng.
7. **Môi trường.** Project sử dụng C++17 trên môi trường UNIX. Sinh viên cần bảo đảm năm file nộp có thể biên dịch cùng starter gốc mà không yêu cầu thêm header hoặc source file tự tạo bên ngoài.

# 7 Nộp bài

Sinh viên chỉ nộp **năm file** dưới đây:

- `BST.h`
- `AVL.h`
- `Heap.h`
- `BKMove.h`
- `BKMove.cpp`

Thời hạn, cách đóng gói và nơi nộp chính thức được thông báo trên LMS. Hệ thống chấm không sử dụng các file bổ sung ngoài danh sách trên.

# 8 Các quy định khác

- Không được dùng các container có sẵn như `std::map`, `std::set`, `std::unordered_map`, `std::priority_queue` hoặc các cấu trúc tương đương để thay thế BST, AVL hay Heap. Không dùng các thuật toán Heap có sẵn của STL để thay cho phần cần hiện thực.
- `std::vector` và `std::string` được phép dùng. Sinh viên tự quản lý những đối tượng cấp phát động; chương trình có thể được kiểm tra memory leak, dangling pointer và double-free.
- Public tests chỉ là bộ kiểm thử minh họa; bài nộp còn phải đáp ứng đúng các yêu cầu API, độ phức tạp và trường hợp biên của đặc tả này.
- Sinh viên chịu trách nhiệm về nội dung và tính độc lập của bài nộp; các trường hợp vi phạm liêm chính học thuật được xử lý theo quy định môn học và nhà trường.

# 9 Lịch sử thay đổi

Phiên bản 1.0 là bản phát hành đầu tiên. Mọi điều chỉnh sau đó sẽ được công bố trên LMS và ghi nhận bằng số phiên bản mới.

**HẾT**